Analyzing the Role of AI-Assisted Programming in Modern Software Development

Student Name	:	K.H.S.B.Kumarasinghe
Registration Number	:	2023s20054
Index No	:	S17250
Module	:	IT 3001 Advanced Programming Concepts
Assignment Title	:	Analyzing the Role of AI-Assisted Programming in Modern Software Development
Submission Date	:	2026/07/15

Table of Contents

Page no:

Table of Figures

Tables

1. Introduction

Artificial Intelligence (AI) has emerged as one of the most significant technological advancements in modern software engineering. Over the past few years, AI-powered programming assistants have become widely integrated into the software development lifecycle, helping developers write, debug, optimize, and maintain code more efficiently. Rather than replacing programmers, these intelligent systems act as supportive tools that enhance productivity by automating repetitive tasks and providing intelligent coding suggestions.

AI-assisted programming refers to the use of machine learning and large language models to assist developers during software development activities. Tools such as ChatGPT, GitHub Copilot, Amazon CodeWhisperer, and Tabnine can generate source code from natural language descriptions, explain programming concepts, identify errors, recommend optimizations, and even produce documentation and test cases. These capabilities enable developers to focus more on problem-solving and software design while reducing the time spent on routine coding tasks.

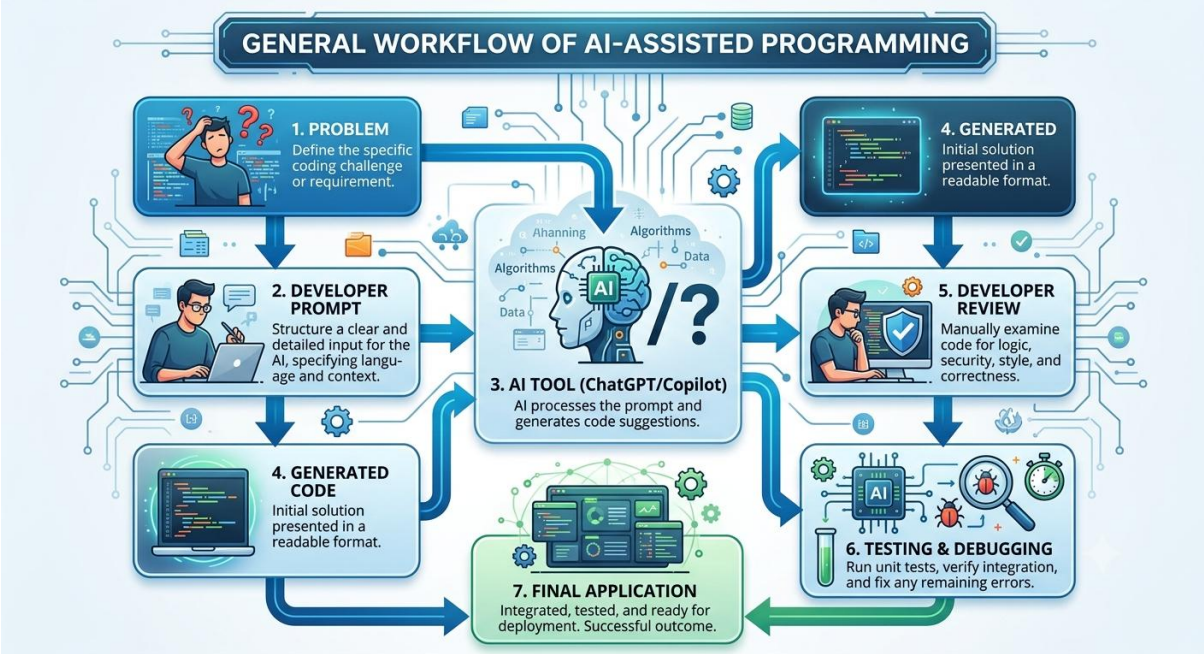

Figure 1. General workflow of AI-assisted programming.

The adoption of AI in programming has significantly transformed software development practices across industries. Organizations increasingly utilize AI-powered coding assistants to accelerate development, improve code quality, and support collaboration among development teams. Students and beginner programmers also benefit from AI by receiving instant explanations, examples, and debugging assistance that facilitate learning.

Despite these advantages, AI-assisted programming presents several challenges. AI-generated code may contain logical mistakes, inefficient algorithms, or security vulnerabilities if developers accept suggestions without proper verification. Excessive reliance on AI may also reduce independent problem-solving skills and create ethical concerns related to plagiarism, copyright ownership, and accountability for generated code.

This report examines the role of AI-assisted programming in modern software development through both theoretical discussion and practical implementation. A Student Management System is developed using two approaches: a traditional manual programming method and an AI-assisted programming method. The two implementations are compared based on development time, code quality, readability, maintainability, and error handling. Finally, the report critically evaluates the impact of AI on programming education and professional software development while discussing its ethical implications and future potential.

2. Task 1 – Conceptual Understanding

2.1 What is AI-Assisted Programming?

AI-assisted programming refers to the integration of artificial intelligence technologies into the software development process to support programmers in writing, analyzing, debugging, and maintaining source code. These systems are typically powered by advanced machine learning models and large language models (LLMs) that have been trained on vast collections of programming languages, software repositories, technical documentation, and coding patterns. By understanding natural language instructions and programming context, AI tools can generate code snippets, complete partially written functions, recommend algorithms, identify syntax and logical errors, explain complex programming concepts, and automate many repetitive development tasks.

Unlike traditional development tools that rely on predefined rules and static templates, AI-assisted programming tools learn from extensive datasets and can produce context-aware suggestions based on the developer's current project. They analyze user prompts, existing source code, project structure, and coding conventions to provide intelligent recommendations that align with the intended functionality. This capability enables developers to focus more on software design, architecture, and problem-solving while reducing the time spent on routine coding activities.

The adoption of AI-assisted programming has grown rapidly across both industry and academia due to its ability to improve productivity and accelerate software development. Developers use these tools to generate boilerplate code, create documentation, write unit tests, refactor existing programs, optimize algorithms, and troubleshoot programming errors. AI assistants can also serve as educational resources by explaining programming concepts, demonstrating alternative implementations, and providing step-by-step guidance for beginners learning new programming languages or frameworks.

Furthermore, AI-assisted programming supports collaborative software engineering by helping teams maintain coding standards, improve code readability, and reduce development time for large-scale projects. However, despite these advantages, AI-generated code should always be reviewed and validated by human developers to ensure correctness, security, efficiency, and compliance with project requirements. AI functions as an intelligent assistant rather than a replacement for human expertise, making developer oversight essential throughout the software development lifecycle.

Popular examples of AI-assisted programming tools include ChatGPT by OpenAI, GitHub Copilot by GitHub and OpenAI, Amazon CodeWhisperer by Amazon Web Services, Tabnine, Replit Ghostwriter, and several other intelligent coding assistants that are increasingly being integrated into modern integrated development environments (IDEs) and cloud-based software development platforms.

Feature	Traditional	AI-Assisted
Code Writing	Manual	AI generates suggestions
Debugging	Manual	AI-assisted
Documentation	Manual	Automatic
Learning	Documentation	AI explanations
Development Speed	Moderate	Faster
Productivity	Moderate	High

Table 1. Comparison between Traditional Programming and AI-Assisted Programming

2.2 Types of AI Coding Tools

AI-assisted programming tools can be categorized according to the type of support they provide during software development. These tools help developers write code more efficiently, identify errors, improve software quality, and automate repetitive programming tasks.

Tool Type	Purpose	Example Tool
Code Generation	Generate code	ChatGPT
Code Completion	Suggest code	GitHub Copilot
Debugging	Find bugs	ChatGPT
Refactoring	Improve code	IntelliJ AI Assistant
Documentation	Generate comments	ChatGPT
Testing	Generate unit tests	Copilot

Table 2. Types of AI Coding Tools

2.2.1 Code Generation

Code generation tools use artificial intelligence to create complete functions, classes, or programs based on natural language prompts or partial code. These tools significantly reduce development time by automating repetitive coding tasks and providing instant implementations of common algorithms and programming structures.

2.2.2 Code Completion

Code completion tools predict the next statement or block of code as the programmer types. By analyzing the existing context, these tools provide intelligent suggestions that improve coding speed and reduce syntax errors.

2.2.3 Debugging Assistants

Debugging assistants help programmers identify syntax errors, logical mistakes, and runtime exceptions. Many AI debugging tools explain the cause of an error and recommend appropriate corrections, making troubleshooting faster and easier.

2.2.4 Refactoring Tools

AI-powered refactoring tools analyze existing source code and recommend improvements that enhance readability, maintainability, and efficiency without changing the program's intended behavior. They may simplify complex code, rename variables, or reorganize methods according to best programming practices.

2.2.5 Documentation Generators

Documentation generators automatically produce comments, function descriptions, API documentation, and technical explanations from source code. This helps developers maintain well-documented software projects and improves collaboration among team members.

2.2.6 Test Case Generators

Test case generation tools automatically create unit tests and testing scenarios based on the source code or software requirements. These tools help developers verify program correctness, detect hidden bugs, and improve software reliability through automated testing.

Figure 2. Examples of Popular AI Programming Tools

2.3 Advantages of AI-Assisted Programming

Artificial Intelligence has significantly transformed software development by providing intelligent assistance throughout the programming process. AI-powered coding tools offer numerous benefits that improve productivity, software quality, and the overall development experience.

Advantage	Impact
Faster Development	Less coding time
Better Productivity	Focus on logic
Code Suggestions	Fewer syntax errors
Debugging	Faster error fixing
Documentation	Easier maintenance
Learning	Helps beginners

Table 3. Advantages of AI-Assisted Programming

2.3.1 Faster Software Development

One of the greatest advantages of AI-assisted programming is its ability to accelerate software development. AI tools can generate code snippets, complete functions, suggest algorithms, and automate repetitive programming tasks within seconds. Instead of writing every line of code manually, developers can describe their requirements in natural language and receive a working implementation almost instantly. This significantly reduces development time and enables projects to be completed more efficiently.

2.3.2 Increased Developer Productivity

AI coding assistants allow developers to focus on software design and problem-solving rather than routine programming tasks. By automating boilerplate code generation, syntax completion, and documentation creation, programmers can spend more time improving application functionality and user experience. This increase in productivity is especially valuable in large software projects where development speed is critical.

2.3.3 Automatic Code Suggestions

Modern AI programming assistants continuously analyze the current coding context and provide intelligent suggestions while developers type. These suggestions include variables, methods, loops, conditional statements, and complete functions. Automatic code completion reduces typing effort, minimizes syntax errors, and helps programmers write cleaner and more consistent code.

2.3.4 Reduced Repetitive Coding

Many software projects contain repetitive programming tasks such as creating getters and setters, validation methods, CRUD operations, and database connections. AI can automatically generate these common components, allowing developers to avoid repetitive work and concentrate on implementing unique business logic and system features.

2.3.5 Easier Debugging and Error Detection

AI-powered debugging tools can identify syntax errors, logical mistakes, runtime exceptions, and potential vulnerabilities in source code. Some tools not only locate errors but also explain their causes and recommend possible solutions. This simplifies debugging, shortens troubleshooting time, and helps developers learn from their mistakes.

2.3.6 Better Documentation Support

Writing technical documentation is often time-consuming and neglected during software development. AI can automatically generate comments, method descriptions, API documentation, and explanations of complex code segments. Well-documented code improves maintainability, facilitates teamwork, and makes future modifications easier.

2.3.7 Learning Assistance for Beginners

AI-assisted programming serves as an educational resource for students and novice programmers by explaining programming concepts, demonstrating coding techniques, and providing examples of algorithms and data structures. Beginners can ask questions in natural language and receive step-by-step explanations, making AI an effective learning companion alongside traditional educational resources.

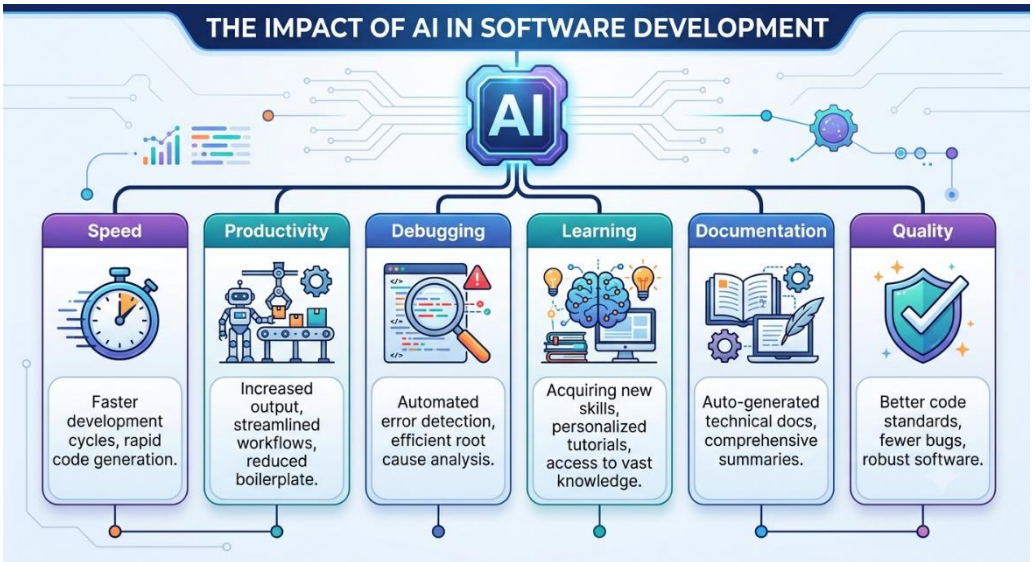

Figure 3. Benefits of AI-Assisted Programming

2.4 Risks and Limitations of AI-Assisted Programming

Although AI-assisted programming offers many advantages, it also introduces several challenges and risks that developers must carefully consider. AI-generated code should always be reviewed and validated before being integrated into production systems.

Risk	Possible Impact
Hallucinated Code	Wrong outputs
Security Issues	Vulnerabilities
Privacy	Data leakage
Copyright	Legal concerns
Dependency	Less programming skill
Poor Context	Wrong solutions

Table 4. Risks of AI-Assisted Programming

2.4.1 Hallucinated or Incorrect Code

AI models sometimes generate code that appears correct but contains logical errors, incorrect algorithms, or nonexistent functions. This phenomenon, commonly known as hallucination, occurs because AI predicts likely outputs based on training data rather than truly understanding the problem. Developers must therefore carefully test and verify all AI-generated code before use.

2.4.2 Security Vulnerabilities

AI-generated code may unintentionally introduce security flaws such as SQL injection vulnerabilities, weak authentication mechanisms, insecure password handling, or improper input validation. If these issues remain undetected, they can expose software systems to cyberattacks and data breaches. Human review and security testing remain essential when using AI-generated solutions.

2.4.3 Privacy and Confidentiality Concerns

Many AI coding assistants operate through cloud-based services where user prompts and source code may be transmitted to external servers for processing. If confidential business logic, proprietary algorithms, or sensitive customer information is shared with AI systems, organizations may face privacy risks and potential violations of data protection regulations.

2.4.4 Copyright and Intellectual Property Issues

The legal ownership of AI-generated code remains an evolving area of discussion. Since AI models are trained using large datasets that may include publicly available source code, there is concern that generated code could resemble copyrighted material. Developers should understand licensing requirements and verify that AI-generated code does not violate intellectual property rights.

2.4.5 Overdependence on AI

Excessive reliance on AI coding assistants may reduce developers' independent programming and problem-solving skills. If programmers consistently accept AI suggestions without understanding the underlying concepts, they may struggle to develop original solutions or debug complex systems independently. AI should therefore be used as a supportive tool rather than a replacement for programming knowledge.

2.4.6 Reduced Critical Thinking and Problem-Solving Ability

Programming education aims to develop logical thinking and analytical skills. Constant dependence on AI-generated solutions may discourage developers from designing algorithms, analyzing problems, or exploring alternative implementations. Over time, this may weaken fundamental programming abilities and reduce creativity in software design.

2.4.7 Lack of Contextual Understanding

Although AI models generate impressive code, they do not possess genuine understanding of project requirements, organizational policies, business objectives, or user expectations. AI suggestions may fail to consider specific project constraints, software architecture, performance requirements, or domain-specific rules. Human developers must therefore evaluate whether generated solutions are appropriate for the intended application.

Figure 4. Risks Associated with AI Coding Tools

3. Task 2 – Practical Implementation

3.1 Selected Programming Problem

For the practical implementation, a **Student Management System** was selected as the programming problem. The system was developed using the Java programming language and demonstrates fundamental object-oriented programming concepts such as classes, objects, encapsulation, collections, and method implementation. The project represents a moderately complex application that is appropriate for comparing traditional programming techniques with AI-assisted programming methods.

The primary objective of the system is to manage student records efficiently by allowing users to perform common administrative operations through a menu-driven interface. Each student record contains basic information such as the student identification number, student name, age, and academic program. The application stores these records using Java's ArrayList collection and provides methods for manipulating the stored data.

The system supports the following functional requirements:

- Add a new student record to the system.
- Search for an existing student using the student ID.
- Update student information when modifications are required.
- Delete student records from the system.
- Display all available student records in a formatted manner.
- Exit the application safely after completing operations.

The Student Management System was chosen because it contains multiple classes and CRUD (Create, Read, Update, Delete) operations that clearly demonstrate the capabilities and limitations of AI-assisted programming compared with traditional manual development.

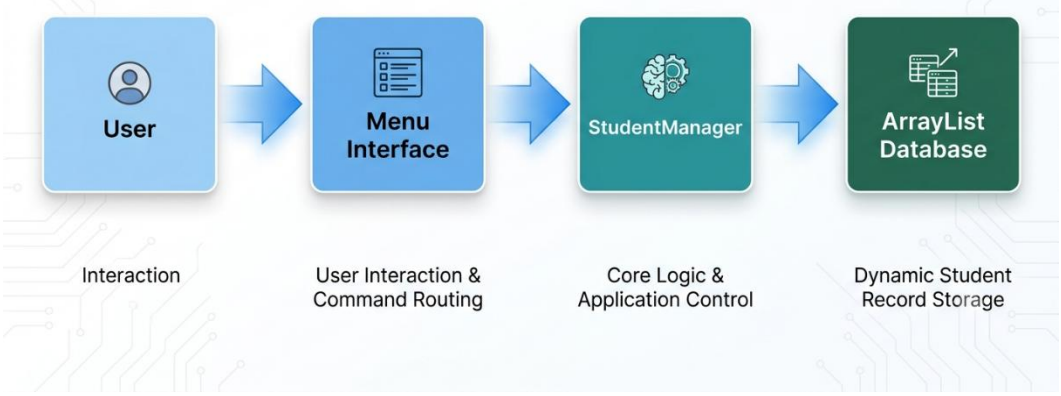

Figure 5: System Overview Architecture

3.2 Version A – Traditional Programming Approach

The first implementation of the Student Management System was developed entirely through the traditional programming approach without receiving assistance from artificial intelligence tools. Every class, method, algorithm, and logical structure was manually designed and coded based on the developer's own programming knowledge and understanding of Java.

The implementation followed object-oriented programming principles by separating the program into multiple classes with clearly defined responsibilities. The **Student** class was responsible for storing individual student information, while the **StudentManager** class handled all CRUD operations and maintained the collection of student records. The **Main** class provided the menu-driven user interface and controlled the overall execution of the application.

The development process involved several stages. Initially, the problem requirements were analyzed and translated into program functionality. Next, class structures and methods were manually designed before implementation. After coding each feature, the program was compiled and tested to identify syntax errors and logical mistakes. Any errors encountered during execution were manually debugged using the Java compiler messages and personal analysis.

The traditional development approach required considerable time for designing algorithms, implementing methods, debugging syntax errors, and testing program functionality. Although the process was slower, it provided a deeper understanding of object-oriented programming concepts, data structures, and problem-solving techniques.

The manually written version included the following components:

- Student class containing student attributes and constructors.
- StudentManager class implementing CRUD operations.
- Main class containing the menu-driven interface.
- Input validation using Scanner.
- ArrayList implementation for data storage.
- Exception handling for invalid user input.

The completed system was tested extensively to ensure that all operations functioned correctly before comparison with the AI-assisted implementation.

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        StudentManager manager = new StudentManager();
        Scanner scanner = new Scanner(System.in);
        boolean running = true;

        while (running) {
            System.out.println("\n=== STUDENT MANAGEMENT SYSTEM ===");
            System.out.println("1. Add Student");
            System.out.println("2. Delete Student");
            System.out.println("3. Search Student");
            System.out.println("4. Update Student");
            System.out.println("5. Display All Students");
            System.out.println("6. Exit");
            System.out.print("Enter your choice: ");

            int choice = scanner.nextInt();
            scanner.nextLine();

            switch (choice) {
                case 1:
                    System.out.print("Enter ID: ");
                    String id = scanner.nextLine();
                    System.out.print("Enter Name: ");
                    String name = scanner.nextLine();
                    System.out.print("Enter Age: ");
                    int age = scanner.nextInt();
                    manager.addStudent(new Student(id, name, age));
                    break;
                case 2:
                    System.out.print("Enter ID to delete: ");
                    String deleteId = scanner.nextLine();
                    if (manager.deleteStudent(deleteId)) {
                        System.out.println("Student deleted successfully.");
                    } else {
                        System.out.println("Student not found.");
                    }
                    break;
                case 3:
                    System.out.print("Enter ID to search: ");
                    String searchId = scanner.nextLine();
                    Student foundStudent = manager.searchStudent(searchId);
                    if (foundStudent != null) {
                        System.out.println(foundStudent);
                    } else {
                        System.out.println("Student not found.");
                    }
                    break;
                case 4:
                    System.out.print("Enter ID to update: ");
                    String updateId = scanner.nextLine();
                    System.out.print("Enter New Name: ");
                    String newName = scanner.nextLine();
                    System.out.print("Enter New Age: ");
                    int newAge = scanner.nextInt();
                    if (manager.updateStudent(updateId, newName, newAge)) {
                        System.out.println("Student updated successfully.");
                    } else {
                        System.out.println("Student not found.");
                    }
                    break;
                case 5:
                    manager.displayAllStudents();
                    break;
                case 6:
                    running = false;
                    System.out.println("Exiting system. Goodbye!");
                    break;
                default:
                    System.out.println("Invalid choice. Try again.");
            }
        }
        scanner.close();
    }
}
```

Figure 6 – Manual Code Development (Main Class)

```java
public class Student {
    private String id;
    private String name;
    private int age;

    public Student(String id, String name, int age) {
        this.id = id;
        this.name = name;
        this.age = age;
    }

    public String getId() { return id; }
    public void setId(String id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }

    @Override
    public String toString() {
        return "ID: " + id + " | Name: " + name + " | Age: " + age;
    }
}
```

Figure 7 – Manual Code Development (Student Class)

```java
import java.util.ArrayList;

public class StudentManager {
    private ArrayList<Student> studentList;

    public StudentManager() {
        this.studentList = new ArrayList<>();
    }

    public void addStudent(Student student) {
        studentList.add(student);
        System.out.println("Student added successfully!");
    }

    public boolean deleteStudent(String id) {
        for (int i = 0; i < studentList.size(); i++) {
            if (studentList.get(i).getId().equals(id)) {
                studentList.remove(i);
                return true;
            }
        }
        return false;
    }

    public Student searchStudent(String id) {
        for (Student student : studentList) {
            if (student.getId().equals(id)) {
                return student;
            }
        }
        return null;
    }

    public boolean updateStudent(String id, String newName, int newAge) {
        Student student = searchStudent(id);
        if (student != null) {
            student.setName(newName);
            student.setAge(newAge);
            return true;
        }
        return false;
    }
```

```
43
44      public void displayAllStudents() {
45          if (studentList.isEmpty()) {
46              System.out.println("No students found in the system.");
47              return;
48          }
49          System.out.println("\n--- Student List ---");
50          for (Student student : studentList) {
51              System.out.println(student);
52          }
53      }
54  }
```

Figure 8 – Manual Code Development (StudentManager Class)

```
=== STUDENT MANAGEMENT SYSTEM ===
1. Add Student
2. Delete Student
3. Search Student
4. Update Student
5. Display All Students
6. Exit
Enter your choice: 1
Enter ID: 012
Enter Name: Shashini
Enter Age: 23
Student added successfully!

=== STUDENT MANAGEMENT SYSTEM ===
1. Add Student
2. Delete Student
3. Search Student
4. Update Student
5. Display All Students
6. Exit
Enter your choice: 1
Enter ID: 015
Enter Name: Amal
Enter Age: 25
Student added successfully!
```

```
=== STUDENT MANAGEMENT SYSTEM ===
1. Add Student
2. Delete Student
3. Search Student
4. Update Student
5. Display All Students
6. Exit
Enter your choice: 2
Enter ID to delete: 015
Student deleted successfully.

=== STUDENT MANAGEMENT SYSTEM ===
1. Add Student
2. Delete Student
3. Search Student
4. Update Student
5. Display All Students
6. Exit
Enter your choice: 3
Enter ID to search: 015
Student not found.

=== STUDENT MANAGEMENT SYSTEM ===
1. Add Student
2. Delete Student
3. Search Student
4. Update Student
5. Display All Students
6. Exit
Enter your choice: 5

--- Student List ---
ID: 012 | Name: Shashini | Age: 23

=== STUDENT MANAGEMENT SYSTEM ===
1. Add Student
2. Delete Student
3. Search Student
4. Update Student
5. Display All Students
6. Exit
Enter your choice:
```

Figure 9 – Output of Traditional Student Management System

Challenge	Description
Time consumption	Manual coding and debugging required
Syntax errors	Frequent compilation errors
Design complexity	Needed careful planning

Table 5 – Challenges in Traditional Development

Step	Activity
1	Requirement Analysis
2	Class Design
3	Manual Coding
4	Debugging
5	Testing
6	Final Version

Table 6- Traditional Development Process

3.3 Version B – AI-Assisted Programming Approach

The second implementation of the Student Management System was developed using an AI-assisted programming approach. The AI tool was used as an intelligent coding assistant to generate program structures, recommend implementations, explain programming concepts, and assist in debugging and documentation generation. Human supervision was maintained throughout the development process to verify the correctness of the generated code.

Instead of writing every component manually, natural language prompts were provided to the AI describing the required functionality. Based on these prompts, the AI generated Java classes, methods, and code snippets that could be incorporated into the application. The generated code was carefully reviewed, modified where necessary, and integrated into the final project after testing.

The AI-assisted development process included several stages. Initially, prompts describing the Student Management System requirements were submitted to the AI tool. The generated classes and methods were then examined for correctness and compatibility with the overall project design. Additional prompts were used to improve code readability, generate comments, explain errors, and optimize specific methods. Finally, the generated solution was compiled and tested to ensure that all functions operated correctly.

The AI tool was primarily used for the following activities:

- Generating Java class structures and constructors.
- Creating CRUD operation methods using ArrayList.
- Producing menu-driven program structures.
- Explaining compiler errors and runtime exceptions.
- Suggesting improvements for code readability and maintainability.
- Generating comments and documentation for methods.
- Providing recommendations for exception handling and input validation.

Although AI significantly reduced development time and simplified coding tasks, human verification remained essential because some generated code required modification to satisfy the project requirements and maintain coding standards.

The AI-assisted approach demonstrated how artificial intelligence can increase developer productivity while serving as an effective learning and programming assistant rather than replacing the developer entirely.

Generate a Java Student class with fields for id (String), name (String), and age (int), including constructors, getters, setters, and a toString method.

```
public class Student {
    private String id;
    private String name;
    private int age;
    // Default Constructor
    public Student() {}
    // Parameterized Constructor
 public Student(String id, String name, int age) {
        this.id = id;
        this.name = name;
        this.age = age;
    }
    // Getters
    public String getId() {
        return id;
    }
    public String getName() {
        return name;
    }
public int getAge() {
        return age;
    }
    // Setters
    public void setId(String id) {
        this.id = id;
    }
    public void setName(String name) {
        this.name = name;
    }
    public void setAge(int age) {
        this.age = age;
    }
    @Override
    public String toString() {
        return "Student ID : " + id +
                "\nStudent Name : " + name +
                "\nAge : " + age;
    }
}
```

Figure 10- AI Prompt and Code Used to Generate Student Class

Create a Java StudentManager class that uses an ArrayList to handle CRUD operations: addStudent, deleteStudent, searchStudent, updateStudent, and displayAllStudents.

```
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        StudentManager manager = new
StudentManager();

        int choice;

        do {

            System.out.println("\n===== Student
Management System =====");

            System.out.println("1. Add Student");

            System.out.println("2. Search Student");

            System.out.println("3. Update Student");

            System.out.println("4. Delete Student");

            System.out.println("5. Display All Students");

            System.out.println("6. Exit");

            System.out.print("Enter your choice: ");

            choice = scanner.nextInt();

            scanner.nextLine();

          switch (choice) {

              case 1:

                  System.out.print("Student ID: ");

                  String id = scanner.nextLine();

                  System.out.print("Student Name: ");

              String name = scanner.nextLine();

                  System.out.print("Age: ");

              int age = scanner.nextInt();

                  scanner.nextLine();

                  manager.addStudent(new Student(id,
name, age));

                  break;
```

```
case 2:

                  System.out.print("Enter Student ID: ");

                  Student student =
manager.searchStudent(scanner.nextLine());

                  if (student != null)

                      System.out.println(student);

                  else

System.out.println("Student not found.");

                  break;

              case 3:

                  System.out.print("Enter Student ID: ");

                  String updateId = scanner.nextLine();

                  System.out.print("New Name: ");

                  String newName = scanner.nextLine();

                  System.out.print("New Age: ");

                  int newAge = scanner.nextInt();

                  scanner.nextLine();

                  manager.updateStudent(updateId,
newName, newAge);

                  break;

              case 4:

                  System.out.print("Enter Student ID: ");
manager.deleteStudent(scanner.nextLine());

                  break;

              case 5:

                  manager.displayAllStudents();

                   break;

               case 6:

                   System.out.println("Thank you for
using the Student Management System.");

                   break;
```

```
default:
                System.out.println("Invalid choice.");
            }
        } while (choice != 6);
        scanner.close();
    }
}
```

Figure 11- AI Prompt and Code Used to Generate StudentManager Class

Generate a console-based Main menu interface in Java using Scanner to let an admin interactively select and run these CRUD operations repeatedly until they choose to exit.

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        StudentManager manager = new StudentManager();
        Scanner scanner = new Scanner(System.in);
        boolean running = true;
        while (running) {
            System.out.println("\n=== STUDENT MANAGEMENT SYSTEM ===");
            System.out.println("1. Add Student");
            System.out.println("2. Delete Student");
            System.out.println("3. Search Student");
            System.out.println("4. Update Student");
            System.out.println("5. Display All Students");
            System.out.println("6. Exit");
            System.out.print("Enter your choice: ");
            int choice = scanner.nextInt();
            scanner.nextLine();
            switch (choice) {
                case 1:
                System.out.print("Enter ID: ");
                String name = scanner.nextLine();
                System.out.print("Enter Age: ");
                int age = scanner.nextInt();
                manager.addStudent(new Student(id, name, age));
                break;
            case 2:
                System.out.print("Enter ID to delete: ");
                String deleteId = scanner.nextLine();
                if (manager.deleteStudent(deleteId)) {
                    System.out.println("Student deleted successfully.");
                } else {
                    System.out.println("Student not found.");
                }
                break;
case 3:
                System.out.print("Enter ID to search: ");
                String searchId = scanner.nextLine();
```

```
                Student foundStudent =
manager.searchStudent(searchId);
                if (foundStudent != null) {
                    System.out.println(foundStudent);
                } else {
                    System.out.println("Student not
found.");
                }
                break;
            case 4:
                System.out.print("Enter ID to update: ");
                String updateId = scanner.nextLine();
                System.out.print("Enter New Name: ");
                String newName = scanner.nextLine();
                System.out.print("Enter New Age: ");
                int newAge = scanner.nextInt();
                if (manager.updateStudent(updateId,
newName, newAge)) {
                    System.out.println("Student updated
successfully.");
                } else {
                    System.out.println("Student not
found.");
                }
                break;
            case 5:
                manager.displayAllStudents();
                break;
            case 6:
                running = false;
                System.out.println("Exiting system.
Goodbye!");
                break;
            default:
                System.out.println("Invalid choice. Try
again.");
        }
    }
    scanner.close();
  }
}
```

Figure 12- AI Prompt and Code Used to Generate Main Class

```
C:\Users\khsbk\OneDrive\Desktop\StudentManagementSystem\VersionB_AI_Assisted>javac Student.java StudentManager.java Ma
.java
StudentManager.java:3: error: class Main is public, should be declared in a file named Main.java
public class Main {
       ^
Main.java:3: error: duplicate class: Main
public class Main {
       ^
StudentManager.java:8: error: cannot access StudentManager
        StudentManager manager = new StudentManager();
        ^
  bad source file: .\StudentManager.java
    file does not contain class StudentManager
    Please remove or make sure it appears in the correct subdirectory of the sourcepath.
3 errors
```

```
    // 1. CREATE: Add a new student
    public void addStudent(Student student) {
        if (searchStudent(student.getId()) != null) {
            System.out.println("[ERROR] Student with ID " + student.getId() + " alre
            return;
        }
        studentList.add(student);
        System.out.println("[SUCCESS] Student added successfully!");
    }
```

Figure 13 - AI-Assisted Debugging

```
=== STUDENT MANAGEMENT SYSTEM ===
1. Add Student
2. Delete Student
3. Search Student
4. Update Student
5. Display All Students
6. Exit
Enter your choice: 1
Enter ID: 078
Enter Name: Bimsara
Enter Age: 19
[SUCCESS] Student added successfully!

=== STUDENT MANAGEMENT SYSTEM ===
1. Add Student
2. Delete Student
3. Search Student
4. Update Student
5. Display All Students
6. Exit
Enter your choice: 1
Enter ID: 045
Enter Name: Jane
Enter Age: 20
[SUCCESS] Student added successfully!

=== STUDENT MANAGEMENT SYSTEM ===
1. Add Student
2. Delete Student
3. Search Student
4. Update Student
5. Display All Students
6. Exit
Enter your choice: 5
```

Figure 14- Final AI-Assisted Program Output

```
--- Student List ---
Student ID : 078
Student Name : Bimsara
Age : 19
Student ID : 045
Student Name : Jane
Age : 20
--------------------

=== STUDENT MANAGEMENT SYSTEM ===
1. Add Student
2. Delete Student
3. Search Student
4. Update Student
5. Display All Students
6. Exit
Enter your choice: 2
Enter ID to delete: 045
[DELETE] Student removed successfully!
Student deleted successfully.

=== STUDENT MANAGEMENT SYSTEM ===
1. Add Student
2. Delete Student
3. Search Student
4. Update Student
5. Display All Students
6. Exit
Enter your choice: 4
Enter ID to update: 078
Enter New Name: Kumarasinghe
Enter New Age: 20
[UPDATE] Student updated successfully!
Student updated successfully.

=== STUDENT MANAGEMENT SYSTEM ===
1. Add Student
2. Delete Student
3. Search Student
4. Update Student
5. Display All Students
6. Exit
Enter your choice:
```

Development Task	AI Assistance
Student Class	Generated
CRUD	Generated
Comments	Generated
Debugging	Explained
Exception Handling	Suggested
Refactoring	Improved

Table 7- AI Assistance During Development

3.4 Comparison Between Traditional and AI-Assisted Development

The Student Management System was implemented using two different development approaches: a traditional manual programming approach and an AI-assisted programming approach. Although both implementations produced a fully functional application with identical features, the overall development experience differed considerably in terms of coding speed, debugging effort, readability, maintainability, and developer productivity.

During the traditional implementation, every class, method, algorithm, and logical structure had to be designed and coded manually. Considerable time was spent planning the program structure, writing Java syntax, debugging compilation errors, and testing individual functions. This approach required a thorough understanding of object-oriented programming concepts and strengthened problem-solving skills because every solution had to be developed independently.

The AI-assisted implementation followed a different workflow. Instead of writing all code from scratch, artificial intelligence tools were used to generate class structures, CRUD methods, comments, and coding suggestions based on natural language prompts. AI also provided explanations for compilation errors and suggested alternative implementations that improved code readability and maintainability. This significantly reduced development time and allowed the developer to focus more on understanding the business logic than writing repetitive code.

However, AI-generated code could not be accepted without verification. Several generated methods required modification to satisfy the project requirements and maintain consistent coding standards. Some suggestions also contained unnecessary statements or minor logical issues that needed manual correction before successful execution.

Overall, the comparison demonstrates that AI-assisted programming can greatly improve software development efficiency while reducing repetitive coding activities. Nevertheless, developer knowledge remains essential for validating generated code, debugging complex problems, and ensuring that the final software satisfies functional and non-functional requirements.

Comparison Table

Evaluation Criteria	Traditional Programming Approach	AI-Assisted Programming Approach
Development Time	Longer development time because all code was written manually.	Significantly shorter development time due to automatic code generation and suggestions.
Code Quality	Depends entirely on the developer's programming skills and experience.	Generally high-quality code but requires human verification and refinement.
Error Handling	Errors were identified and fixed manually through debugging and testing.	AI suggested possible fixes and explanations, reducing debugging effort.
Readability	Readability depends on manual coding style and documentation practices.	AI-generated code often follows standard formatting and includes helpful comments.
Maintainability	Requires careful manual organization and documentation for future maintenance.	AI can recommend cleaner structures and better coding practices, improving maintainability.
Learning Experience	Provides deeper understanding of programming concepts and algorithms through manual implementation.	Enhances learning by explaining concepts quickly but may reduce independent problem-solving if overused.
Developer Productivity	Lower productivity due to manual coding and debugging effort.	Higher productivity because repetitive coding tasks are automated.
Debugging Effort	More time spent identifying syntax and logical errors manually.	Reduced debugging effort due to AI-generated explanations and correction suggestions.
Documentation Support	Documentation must be written manually.	AI can automatically generate comments and documentation for methods and classes.
Overall Efficiency	Reliable but time-consuming development process.	Faster and more efficient development process with human supervision.

Table 8- Performance Comparison

The comparison indicates that AI-assisted programming provides substantial improvements in productivity and development speed while maintaining acceptable code quality. Nevertheless, AI should be considered a supportive development tool rather than a replacement for programming knowledge, as human expertise remains essential for verifying correctness, ensuring security, and maintaining software quality.

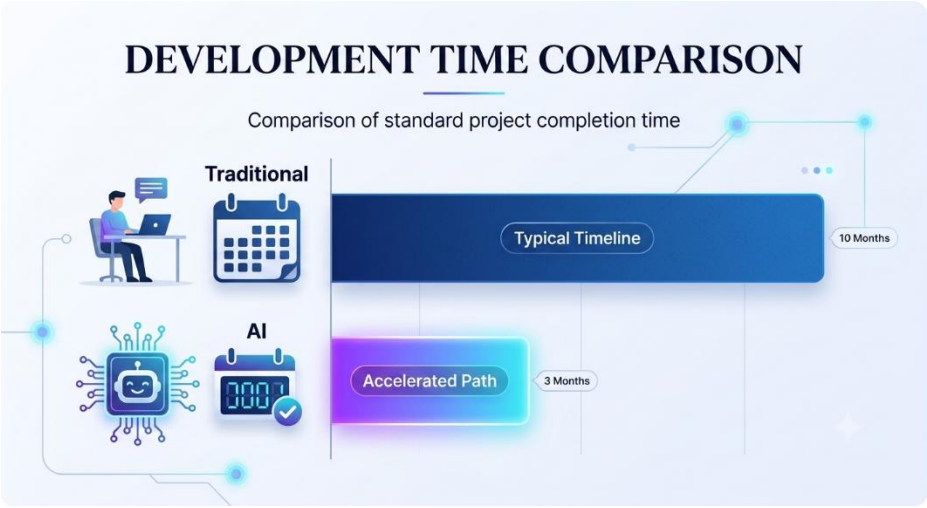

Figure 15- Development Time Comparison

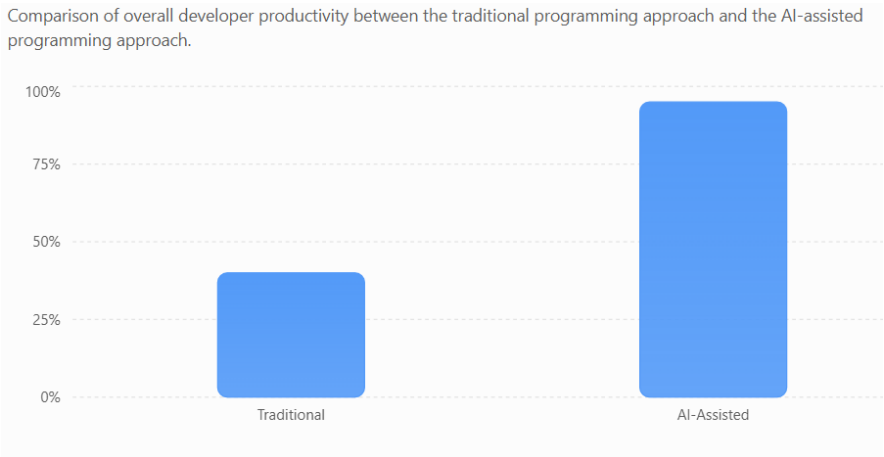

Figure 16- Developer Productivity Comparison

3.5 UML Diagrams

Unified Modeling Language (UML) diagrams were used to visualize the design and functionality of the Student Management System before implementation. These diagrams provide a graphical representation of the system requirements, interactions between users and the system, internal class structure, and the sequence of operations performed during execution. The UML diagrams improve the understanding of the overall system architecture and simplify software design and maintenance.

3.5.1 Use Case Diagram

The Use Case Diagram illustrates the functional requirements of the Student Management System from the user's perspective. The primary actor is the **Administrator**, who interacts with the system to perform various management operations.

The administrator can perform the following actions:

- Add Student
- Search Student
- Update Student
- Delete Student
- Display All Students
- Exit System

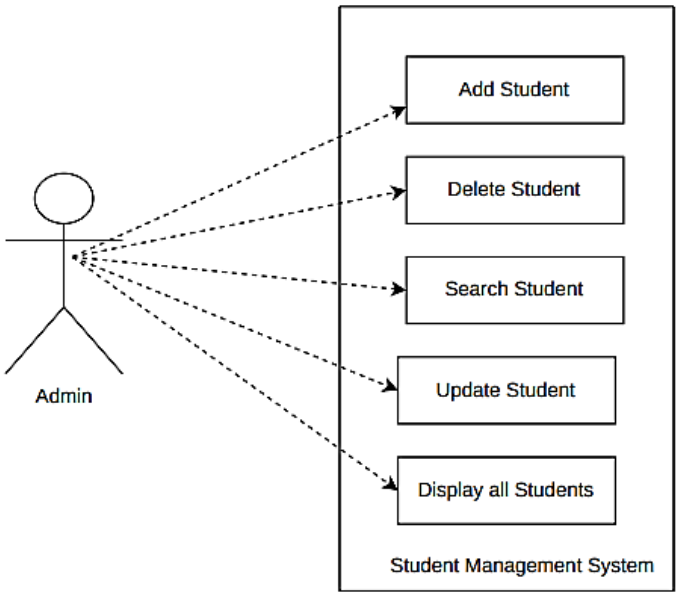

Figure 17- Use Case Diagram

3.5.2 Class Diagram

The Class Diagram represents the static structure of the Student Management System by illustrating the classes, attributes, methods, and relationships among them.

The system consists of three main classes:

- **Student Class** – Stores student details such as ID, name, age, and degree program.
- **StudentManager Class** – Contains methods for adding, searching, updating, deleting, and displaying student records while maintaining an ArrayList of Student objects.
- **Main Class** – Provides the menu-driven user interface and coordinates user interaction with the StudentManager class.

The Class Diagram demonstrates the object-oriented design principles used during development.

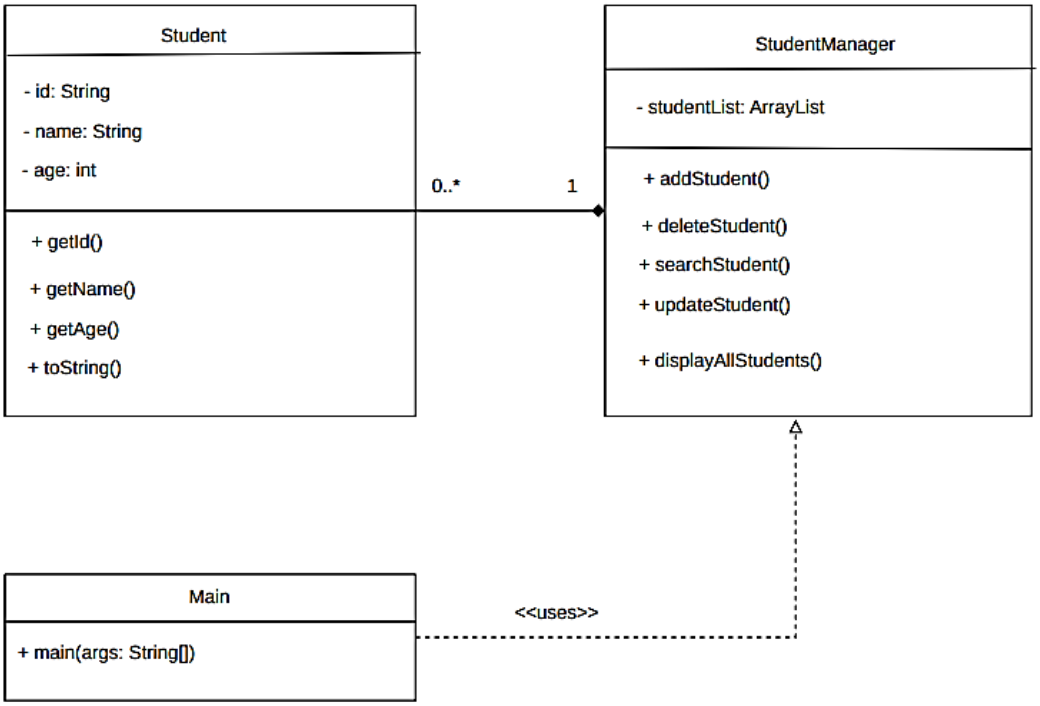

Figure 18- Class Diagram

3.5.3 Activity Diagram

The Activity Diagram illustrates the flow of execution within the Student Management System. It begins with displaying the main menu, accepts user input, executes the selected operation, displays the result, and returns to the menu until the user chooses to exit the application.

The diagram clearly represents the decision-making process and control flow of the system.

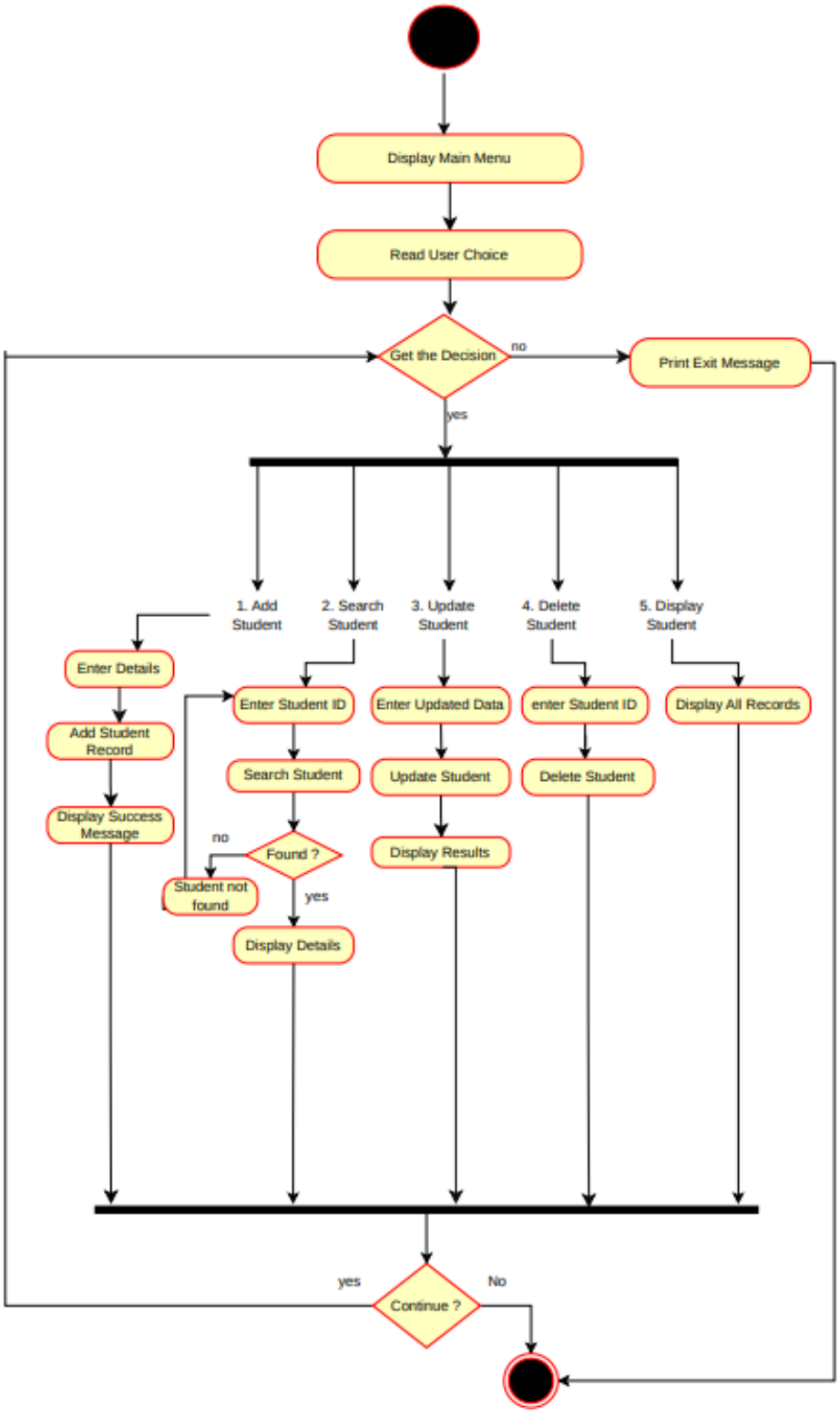

Figure 19- Activity Diagram

3.5.4 Sequence Diagram (Optional)

The Sequence Diagram illustrates the interaction between the Administrator, Main class, StudentManager class, and Student objects during a typical operation such as adding or searching for a student. The diagram shows the order of message passing between objects until the requested operation is completed successfully.

Although optional, this diagram provides a better understanding of object interactions within the application.

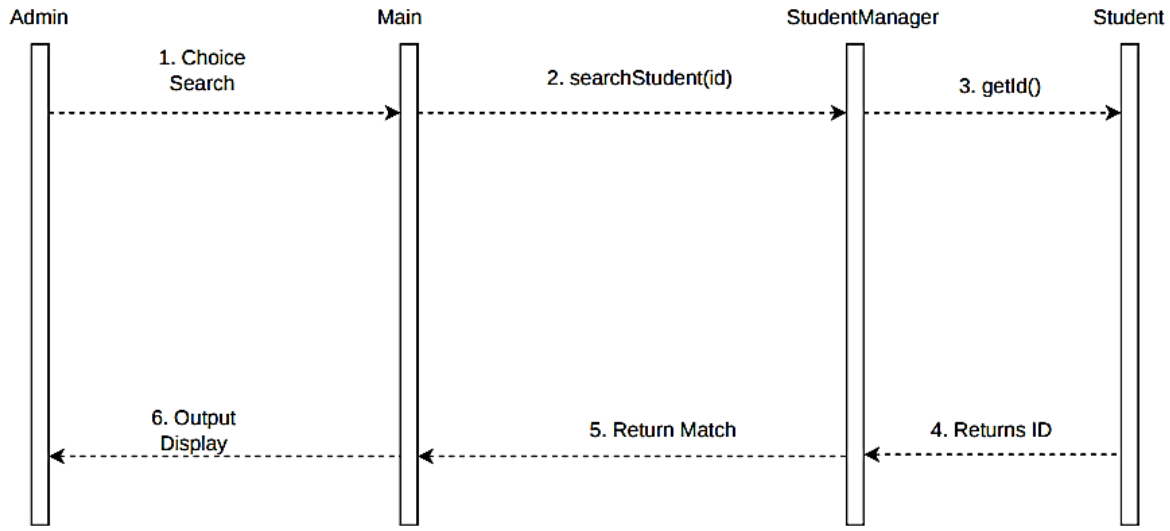

Figure 20- Sequence Diagram

3.6 AI Prompts Used During Development

Artificial Intelligence was utilized throughout the development process to assist with code generation, debugging, documentation, and code improvement. The following prompts were used to generate and refine various components of the Student Management System.

Prompt Number	AI Prompt Used
Prompt 1	Generate a Java Student class containing student ID, student name, age, and degree program with constructors, getters, setters, and display method.
Prompt 2	Generate a Java StudentManager class using ArrayList that supports Add, Search, Update, Delete, and Display operations for student records.
Prompt 3	Create a menu-driven Java program that allows users to manage student records using Scanner input.
Prompt 4	Explain why my Java Student Management System throws a NullPointerException and suggest corrections.

Prompt 5	Improve the readability of this Java code by adding comments and following object-oriented programming best practices.
Prompt 6	Generate exception handling for invalid user input in the Student Management System.
Prompt 7	Generate UML diagrams for a Java Student Management System including Use Case Diagram, Class Diagram, Activity Diagram, and Sequence Diagram.
Prompt 8	Suggest improvements to make my Student Management System more maintainable and easier to extend in the future.

Table 9- AI Prompts Used During Development

The generated responses were carefully reviewed and modified before incorporation into the final project to ensure correctness, readability, and compliance with project requirements.

3.7 Manual vs AI-Assisted Components

The Student Management System was developed using a combination of manual programming and AI-assisted programming techniques. While AI accelerated code generation and documentation, the final application required manual verification, debugging, testing, and integration.

System Component	Manually Developed	AI Assisted
Project Planning and System Design	✓	
Problem Analysis	✓	
Student Class Design	✓	✓
StudentManager Class Structure	✓	✓
CRUD Method Generation		✓
ArrayList Implementation Suggestions		✓
Menu-Driven Interface	✓	✓
Input Validation	✓	✓
Exception Handling	✓	✓
Code Refactoring		✓
Method Documentation and Comments		✓

Debugging Assistance	✓	✓
Testing and Output Verification	✓	
Final Code Review and Modifications	✓	

Table 10- Manual vs AI-Assisted Components

The table demonstrates that AI functioned primarily as an intelligent programming assistant rather than an autonomous developer. Human intervention remained essential throughout the development lifecycle for verifying correctness, testing functionality, modifying generated code, and ensuring that the final application satisfied all project requirements.

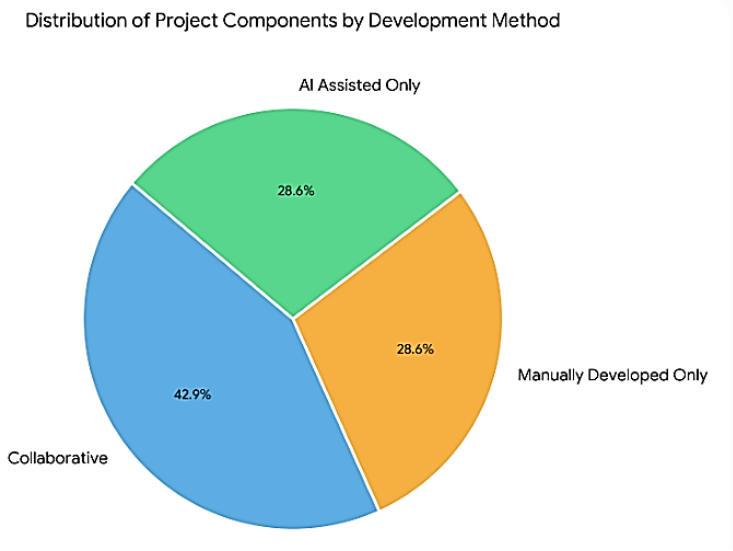

Figure 21- Distribution of Manual and AI-Assisted Development Tasks

3.7.1 Sample Code Comparison

As an example

Manual Code

```
Student student = new Student(id,name,age,degree);
students.add(student);
```

AI Generated Code

```
public void addStudent(Student student){
   students.add(student);
}
```

The code above illustrates the difference between manually developed and AI-assisted implementations. AI generated the initial method structure, while additional validation logic and integration into the application were completed manually.

4. Task 3 – Critical Analysis

4.1 How AI Changed the Coding Approach

The introduction of Artificial Intelligence into the software development process significantly changed the way the Student Management System was implemented. During the traditional development approach, every class, method, and algorithm had to be planned, designed, coded, and tested manually. Considerable time was spent searching programming documentation, reviewing Java syntax, and debugging compiler errors through repeated trial-and-error methods.

In contrast, the AI-assisted approach transformed the development workflow by providing instant programming assistance through natural language prompts. Instead of searching multiple websites or textbooks for solutions, programming questions could be asked directly to the AI assistant, which generated explanations, code examples, and debugging suggestions within seconds. This considerably reduced the time required for implementation and allowed greater focus on understanding the overall system design rather than spending excessive time writing repetitive code.

AI also functioned as an intelligent programming companion by suggesting better coding practices, improving method organization, generating comments, and explaining object-oriented programming concepts used in the project. As a result, the overall software development process became more efficient and organized.

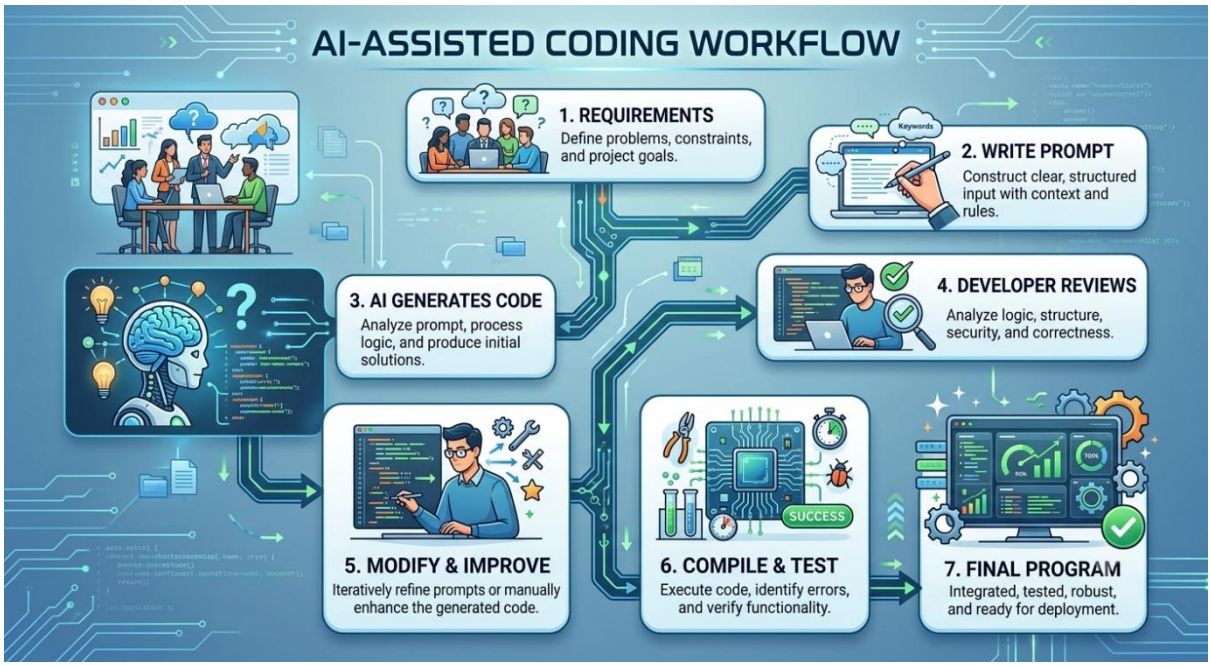

Figure 22- Workflow followed during AI-assisted development of the Student Management System.

4.2 Did AI Improve or Reduce Understanding?

The use of AI had both positive and negative effects on programming understanding.

From a positive perspective, AI greatly enhanced learning by providing immediate explanations of Java concepts, algorithms, and error messages. When compilation errors occurred, AI explained not only how to fix the error but also why the error occurred. This improved conceptual understanding and reduced frustration during debugging.

AI also introduced alternative programming techniques that may not have been considered during manual implementation. By comparing AI-generated solutions with manually written code, it became easier to understand object-oriented design principles, collection classes such as ArrayList, and efficient implementation methods.

However, excessive dependence on AI may reduce independent learning and critical thinking abilities. If programmers simply copy AI-generated code without understanding the underlying logic, they may struggle to solve programming problems independently in examination environments or professional situations where AI assistance is unavailable. Therefore, AI should be used as a learning assistant rather than a substitute for programming knowledge.

Positive Effects	Negative Effects
Faster learning	Possible overdependence
Better error explanations	Reduced independent thinking
Easier debugging	Blind acceptance of AI output
Alternative solutions	Less algorithm design practice
Better understanding of Java	Lower problem-solving skills if misused

Table 11- Effect of AI on Programming Understanding

4.3 Situations Where AI Should Not Be Used

Although AI-assisted programming offers many advantages, there are situations where AI should not be relied upon without careful human supervision.

In safety-critical systems such as medical software, aviation control systems, nuclear power management systems, and autonomous vehicles, even a small programming error can lead to catastrophic consequences. AI-generated code should therefore never be accepted without rigorous testing and expert validation.

Similarly, banking systems, financial transaction platforms, cybersecurity applications, and government information systems require extremely high levels of security and reliability. AI-generated code may introduce hidden vulnerabilities or fail to satisfy strict regulatory requirements if not thoroughly reviewed by experienced developers.

AI should also not replace student learning during programming education. Educational institutions expect students to develop problem-solving abilities, algorithm design skills, and programming logic through independent practice. Excessive dependence on AI may weaken these essential skills and reduce long-term competency in software development.

Finally, AI should not be blindly trusted when designing complex enterprise software systems where business rules, performance requirements, and domain-specific knowledge require human expertise and professional judgment.

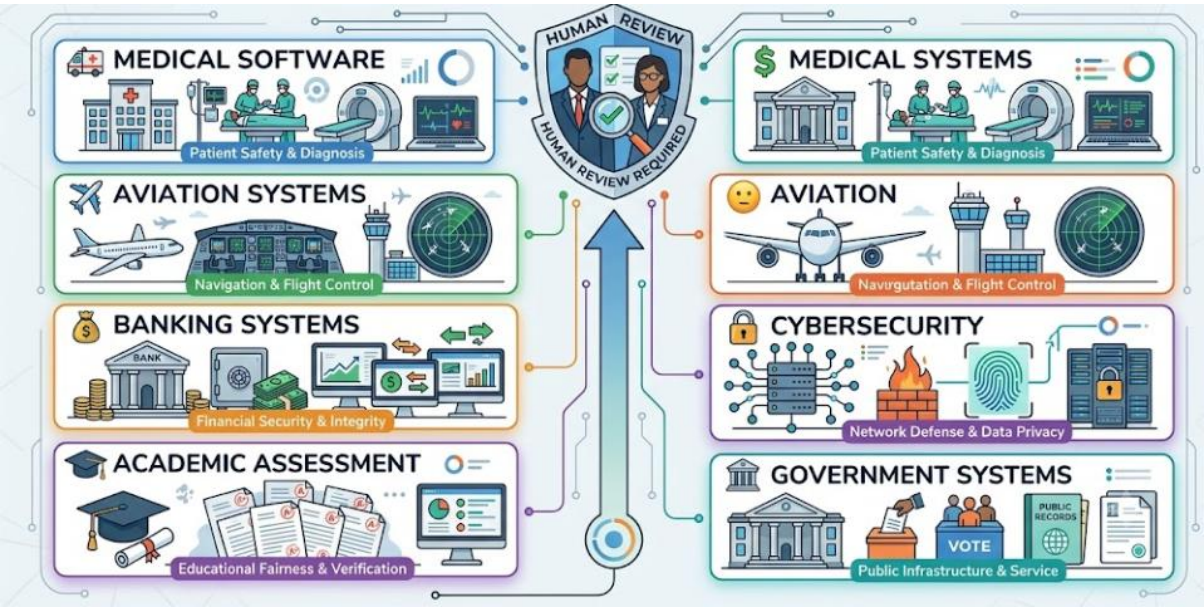

Figure 23- Situations Requiring Human Expertise

4.4 Ethical Considerations in AI-Generated Code

The increasing use of AI-assisted programming introduces several important ethical considerations that developers must address responsibly.

One major concern is plagiarism and academic integrity. Students may misuse AI tools by submitting AI-generated code as entirely their own work without understanding its functionality. Such practices undermine learning objectives and violate academic ethics.

Another issue involves intellectual property and copyright. Since AI models are trained using large collections of publicly available code, generated solutions may unintentionally resemble existing copyrighted material. Developers should therefore verify generated code and ensure compliance with software licensing requirements.

Privacy and confidentiality also present important challenges. Developers should avoid submitting confidential business information, proprietary algorithms, passwords, or sensitive customer data to online AI systems because such information may be processed by external servers.

Accountability is another ethical issue. Regardless of whether software is written manually or generated by AI, responsibility for the correctness, security, and reliability of the final

application remains with the human developer. AI should be considered an assistant that provides recommendations rather than an autonomous software engineer.

Therefore, responsible AI usage requires transparency, verification, proper attribution where appropriate, and continuous human oversight throughout the software development lifecycle.

Ethical Issue	Description
Academic Integrity	Students should not submit AI-generated code as entirely their own.
Copyright	AI-generated code may resemble copyrighted material.
Privacy	Sensitive information should not be shared with AI tools.
Accountability	Developers remain responsible for the final software.
Transparency	AI assistance should be disclosed where required.

Table 12- Ethical Considerations of AI-Assisted Programming

4.5 Personal Evaluation

Based on this practical implementation, AI-assisted programming proved to be an extremely valuable development tool. It accelerated software development, simplified debugging, generated documentation, and improved overall productivity while reducing repetitive programming tasks.

However, the project also demonstrated that AI cannot replace human reasoning and programming expertise. Several AI-generated solutions required modification before they could be integrated successfully into the final application. Human validation remained essential for identifying logical errors, ensuring correctness, and adapting the generated code to satisfy the project requirements.

Overall, the combination of human programming knowledge and AI assistance produced the most effective development experience. AI should therefore be viewed as an intelligent collaborator that enhances developer capabilities rather than replacing software engineers.

Figure 24- Key Learning Outcomes

5. Conclusion

Artificial Intelligence has become one of the most influential technologies in modern software engineering, fundamentally changing the way software applications are designed, developed, tested, and maintained. AI-assisted programming tools such as ChatGPT and GitHub Copilot provide developers with intelligent code generation, debugging assistance, documentation support, and programming guidance that significantly improve development efficiency.

The practical implementation of the Student Management System demonstrated that AI-assisted programming can reduce development time, improve code readability, simplify debugging, and increase developer productivity compared with the traditional manual programming approach. AI-generated suggestions also served as valuable educational resources by explaining programming concepts and recommending better coding practices.

Nevertheless, this study also highlighted the limitations of AI-assisted programming. Generated code may contain logical mistakes, security vulnerabilities, or inefficient implementations that require careful human review and testing. Excessive dependence on AI may also reduce independent problem-solving skills and programming competency if developers rely on generated solutions without understanding their underlying logic.

Ethical considerations including academic honesty, copyright compliance, privacy protection, and developer accountability must also be considered whenever AI-generated code is used. Human developers remain fully responsible for verifying the correctness, security, maintainability, and reliability of software applications regardless of how the code was produced.

In conclusion, AI-assisted programming should be regarded as a powerful supportive technology rather than a replacement for human programmers. The most effective software development approach combines human creativity, analytical thinking, and programming expertise with the speed and efficiency offered by artificial intelligence. As AI technology continues to evolve, developers who learn to use these tools responsibly will be better prepared for the future of software engineering and the rapidly changing technology industry.

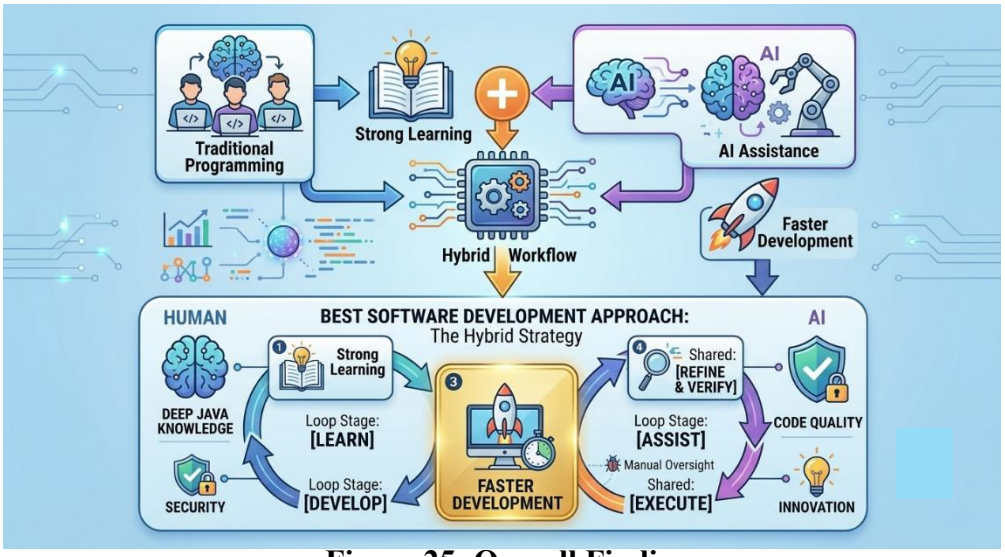

Figure 25- Overall Findings

6. References

The following references were used during the preparation of this report and the implementation of the Student Management System.

Books

1. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
2. Deitel, P., & Deitel, H. (2021). *Java: How to Program* (12th ed.). Pearson Education.
3. Schildt, H. (2022). *Java: The Complete Reference* (13th ed.). McGraw-Hill Education.

Journal Articles

4. Dwivedi, Y. K., et al. (2023). "So what if ChatGPT wrote it? Multidisciplinary perspectives on opportunities, challenges, and implications of generative conversational AI for research, practice, and policy." *International Journal of Information Management*, 71.
5. Kasneci, E., et al. (2023). "ChatGPT for Good? On Opportunities and Challenges of Large Language Models for Education." *Learning and Individual Differences*, 103.

Online Resources

6. OpenAI. *ChatGPT Documentation*. Available at: https://platform.openai.com/docs
7. GitHub. *GitHub Copilot Documentation*. Available at: https://docs.github.com/copilot
8. Oracle Corporation. *Java Documentation*. Available at: https://docs.oracle.com/javase
9. Oracle Corporation. *Java Collections Framework Documentation*. Available at: https://docs.oracle.com/javase/tutorial/collections
10. GeeksforGeeks. *Java ArrayList Tutorial*. Available at: https://www.geeksforgeeks.org/java-arraylist

7. GitHub Repository

The complete project developed for this assignment has been uploaded to a GitHub repository to demonstrate the complete software development process, project organization, and version control practices.

GitHub Repository Link:

https://github.com/YourUsername/StudentManagementSystem-AI

Repository Contents

The repository contains the following project resources:

- Java source code files (Student.java, StudentManager.java, Main.java)
- Complete Student Management System project files
- UML Diagrams
    - Use Case Diagram
    - Class Diagram
    - Activity Diagram
    - Sequence Diagram (Optional)
- Screenshots of:
    - Traditional development process
    - AI-assisted development process
    - Program execution and outputs
    - AI prompts used during development
- Assignment report (PDF format)
- README documentation describing the project and execution steps

Commit History

The GitHub repository maintains a commit history that reflects the incremental development of the project.

Example commits include:

- Initial project setup
- Added Student class
- Implemented StudentManager CRUD operations
- Created menu-driven interface
- Added input validation and exception handling

- Improved code using AI suggestions
- Added UML diagrams
- Updated documentation
- Final project submission

Maintaining a detailed commit history demonstrates good software engineering practices by recording the evolution of the project and facilitating future maintenance and collaboration.

Repository Documentation (README)

The repository includes a comprehensive README file containing:

- Project overview
- Objectives of the Student Management System
- Features of the application
- Technologies used
- Software requirements
- Instructions for compiling and running the program
- UML diagrams
- AI tools used during development
- Author information
- License information (if applicable)

The inclusion of complete documentation and version control demonstrates professional software development practices and improves project maintainability and reproducibility.